

МИНОБРНАУКИ РОССИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования «Тульский государственный университет»
Институт прикладной математики и компьютерных наук

ЛАБОРАТОРНАЯ РАБОТА №4
по дисциплине
Методы и средства защиты информации
по теме
«Количественная оценка стойкости парольной защиты»

Выполнил: ст. гр. 220032-11

Лаукерт К. М.

Принял: к.т.н., доцент ИПМКН

Кулешова Н. В.

Тула 2026

Лабораторная работа №4

по теме «Количественная оценка стойкости парольной защиты»

Цель работы

Программно реализовать генератор паролей, обладающих требуемой стойкостью к взлому, и выполнить количественную оценку их защищённости.

Постановка задачи

Для варианта 9 необходимо использовать русские прописные и строчные буквы, при $P = 10^{-4}$, $V = 3$ пароля в минуту и $T = 15$ суток. Требуется определить мощность алфавита и минимальную длину, реализовать генератор с интерфейсом, рассчитать энтропийные характеристики, сравнить пароли длиной 10 и 12 символов, построить графики и проверить пароль на паттерны и утечки.

Расчёт параметров

Русский алфавит содержит 33 прописные и 33 строчные буквы, поэтому $A = 66$. Срок действия $T = 15 \times 24 \times 60 = 21600$ минут. Нижняя граница числа паролей: $S^* = \text{ceil}(V \times T / P) = \text{ceil}(3 \times 21600 / 10^{-4}) = 648000000$.

Из неравенства $66^L \geq 648000000$ получаем расчётную минимальную длину $L = 5$. При этом минимальное требование методических указаний — не менее 8 символов, поэтому генератор использует $L = 8$. Вероятность подбора для такого пароля: $P = V \times T / 66^8 = 1,800 \times 10^{-10}$, что существенно меньше заданной границы.

Решение

Реализовано нативное desktop-приложение на Python с интерфейсом Tkinter. Случайные символы выбираются модулем secrets, использующим криптографически стойкий генератор. Программа вычисляет нижнюю границу S^* , общую и удельную энтропии, реальную энтропию выборки из 50 паролей, индекс случайности и коэффициент неравномерности. Реализованы проверка распространённых паттернов, сравнительный расчёт для длин 10 и 12 и графики $P(L)$, $P(A)$, $P(V)$, $P(T)$ на Canvas. Для

проверки по базе утечек применяется API Have I Been Pwned по схеме k-anonymity: передаётся только первые пять символов SHA-1-хеша.

Листинг программы (password_gui.py)

```
"""ЛР4. Генератор паролей и количественная оценка стойкости
(вариант 9)."""

import hashlib
import math
import secrets
import tkinter as tk
import urllib.request
from collections import Counter
from tkinter import messagebox, ttk

UPPER = "АБВГДЕЁЖЗИЙКЛМНОПРСТУФХЦЧШЩЪЫЬЭЮЯ"
ALPHABET = UPPER + UPPER.lower()
PATTERNS = ("qwerty", "йцукен", "1234", "abcd", "asdf", "zxcv",
"password")

def password_probability(alphabet_size: int, length: int, speed:
float, days: float) -> float:
    """Вероятность перебора  $P = V * T / A^L$ , где T приводится к
минутам."""
    return min(1.0, speed * days * 24 * 60 /
alphabet_size**length)

def generate_password(length: int, alphabet: str) -> str:
    return "".join(secrets.choice(alphabet) for _ in
range(length))

def find_patterns(password: str) -> list[str]:
    lower = password.lower()
    found = [pattern for pattern in PATTERNS if pattern in
lower]
```

```

        if any(password[index] == password[index + 1] ==
password[index + 2] for index in range(len(password) - 2)):
            found.append("три одинаковых символа подряд")
        if any(password[index:index + 2] == password[index + 2:index
+ 4] for index in range(len(password) - 3)):
            found.append("повторяющаяся группа символов")
    return found

```

```

def sample_metrics(length: int, alphabet: str, count: int = 50)
-> tuple[float, float, float]:
    """Метрики распределения символов по выборке из 50
сгенерированных паролей."""
    sample = "".join(generate_password(length, alphabet) for _
in range(count))
    frequencies = Counter(sample)
    total = len(sample)
    probabilities = [value / total for value in
frequencies.values()]
    real_entropy = -sum(probability * math.log2(probability) for
probability in probabilities)
    unevenness = sum((probability - 1 / len(alphabet)) ** 2 for
probability in probabilities)
    random_index = max(0.0, 1 - real_entropy /
math.log2(len(alphabet)))
    return real_entropy, unevenness, random_index

```

```

class PasswordApp(tk.Tk):
    def __init__(self) -> None:
        super().__init__()
        self.title("ЛР4 – Генератор стойких паролей")
        self.geometry("1000x680")
        self.minsize(900, 620)
        self.configure(padx=16, pady=14)
        self.password = ""
        self.p_var = tk.StringVar(value="0.0001")
        self.v_var = tk.StringVar(value="3")
        self.t_var = tk.StringVar(value="15")

```

```

        self.result_var = tk.StringVar(value="Нажмите
«Сгенерировать и проанализировать».")

        self._build_ui()

        self.analyze()

    def _build_ui(self) -> None:

        ttk.Label(self, text="Генератор стойких паролей",
font=("Segoe UI", 16, "bold")).pack(anchor="w")

        ttk.Label(
            self,
            text="Вариант 9: русские прописные и строчные буквы.
Алфавит А = 66 символов.",
        ).pack(anchor="w", pady=(2, 12))

        parameters = ttk.LabelFrame(self, text="Исходные
данные")

        parameters.pack(fill="x")

        for column, (label, variable) in
enumerate(("Вероятность P:", self.p_var), ("Скорость V,
паролей/мин:", self.v_var), ("Срок T, дней:", self.t_var))):

            ttk.Label(parameters, text=label).grid(row=0,
column=column * 2, padx=(12, 4), pady=10, sticky="w")

            ttk.Entry(parameters, textvariable=variable,
width=16).grid(row=0, column=column * 2 + 1, padx=(0, 12),
pady=10)

            ttk.Button(parameters, text="Сгенерировать и
проанализировать", command=self.analyze).grid(row=0, column=6,
padx=10)

            ttk.Button(parameters, text="Проверить в HIBP",
command=self.check_hibp).grid(row=0, column=7, padx=(0, 12))

        result = ttk.LabelFrame(self, text="Результаты расчёта")

        result.pack(fill="x", pady=12)

        self.result_label = ttk.Label(result,
textvariable=self.result_var, justify="left", font=("Consolas",
10))

        self.result_label.pack(anchor="w", padx=12, pady=10)

        comparison = ttk.LabelFrame(self, text="Сравнение
паролей")

        comparison.pack(fill="x")

```

```

        self.comparison = ttk.Treeview(comparison,
columns=("length", "password", "probability", "entropy"),
show="headings", height=2)

        for column, heading, width in (("length", "Длина", 90),
("password", "Сгенерированный пароль", 270), ("probability",
"Вероятность подбора", 230), ("entropy", "Энтропия", 150)):
            self.comparison.heading(column, text=heading)
            self.comparison.column(column, width=width,
anchor="center")
            self.comparison.pack(fill="x", padx=12, pady=10)

        chart_frame = ttk.LabelFrame(self, text="Графики
зависимостей вероятности подбора")
        chart_frame.pack(fill="both", expand=True, pady=12)
        self.canvas = tk.Canvas(chart_frame, bg="white",
highlightthickness=0)
        self.canvas.pack(fill="both", expand=True, padx=8,
pady=8)
        self.canvas.bind("<Configure>", lambda _event:
self.draw_charts())

        ttk.Label(self, text="Проверка HIBP использует k-
anonymity: передаётся только префикс SHA-1-хеша.",
foreground="#555555").pack(anchor="w")

def parameters(self) -> tuple[float, float, float]:
    try:
        probability = float(self.p_var.get())
        speed = float(self.v_var.get())
        days = float(self.t_var.get())
    except ValueError as error:
        raise ValueError("P, V и T должны быть числами.")
    from error
    if not 0 < probability <= 1 or speed <= 0 or days <= 0:
        raise ValueError("P должно быть в диапазоне (0; 1],
V и T должны быть положительными.")
    return probability, speed, days

def analyze(self) -> None:
    try:

```

```

        target_p, speed, days = self.parameters()
    except ValueError as error:
        messagebox.showerror("Ошибка ввода", str(error))
        return
    total_minutes = days * 24 * 60
    lower_bound = math.ceil(speed * total_minutes /
target_p)
    calculated_length = math.ceil(math.log(lower_bound,
len(ALPHABET)))
    length = max(8, calculated_length)
    self.password = generate_password(length, ALPHABET)
    probability = password_probability(len(ALPHABET),
length, speed, days)
    real_entropy, unevenness, random_index =
sample_metrics(length, ALPHABET)
    patterns = find_patterns(self.password)
    pattern_text = "не обнаружены" if not patterns else ",
".join(patterns)
    self.result_var.set(
        f"Мощность алфавита A: {len(ALPHABET)} символов\n"
        f"Нижняя граница S*: {lower_bound:,} паролей\n"
        f"Расчётная минимальная длина: {calculated_length};
используется: {length} (правило: не менее 8)\n"
        f"Сгенерированный пароль: {self.password}\n"
        f"Вероятность подбора: {probability:.3e}\n"
        f"Общая / удельная энтропия: {length *
math.log2(len(ALPHABET)):.2f} бит /
{math.log2(len(ALPHABET)):.2f} бит/символ\n"
        f"Реальная энтропия выборки / индекс случайности:
{real_entropy:.3f} бит / {random_index:.3f}\n"
        f"Коэффициент неравномерности: {unevenness:.5f}\n"
        f"Паттерны: {pattern_text}"
    )
    self.fill_comparison(speed, days)
    self.draw_charts()

def fill_comparison(self, speed: float, days: float) ->
None:
    for item in self.comparison.get_children():
        self.comparison.delete(item)

```

```

        for length in (10, 12):
            value = password_probability(len(ALPHABET), length,
speed, days)

            self.comparison.insert("", "end", values=(length,
generate_password(length, ALPHABET), f"{value:.3e}", f"{length *
math.log2(len(ALPHABET)):.2f} бит"))

def draw_charts(self) -> None:
    if not self.password:
        return
    try:
        _, speed, days = self.parameters()
    except ValueError:
        return
    canvas = self.canvas
    canvas.delete("all")
    width, height = max(canvas.winfo_width(), 800),
max(canvas.winfo_height(), 320)
    specs = (
        ("P(L)", [password_probability(66, length, speed,
days) for length in range(4, 13)]),
        ("P(A)", [password_probability(size, 12, speed,
days) for size in (10, 20, 30, 40, 50, 66, 80, 94)]),
        ("P(V)", [password_probability(66, 12, value, days)
for value in (1, 3, 10, 30, 100, 300)]),
        ("P(T)", [password_probability(66, 12, speed, value)
for value in (1, 5, 10, 15, 20, 30)]),
    )
    cell_width, cell_height = width // 2, height // 2
    for index, (title, values) in enumerate(specs):
        x0, y0 = (index % 2) * cell_width + 35, (index // 2)
* cell_height + 30
        x1, y1 = (index % 2 + 1) * cell_width - 25, (index
// 2 + 1) * cell_height - 30
        canvas.create_text((x0 + x1) / 2, y0 - 16,
text=title, font=("Segoe UI", 11, "bold"))
        canvas.create_line([x0, y0, x0, y1, x1, y1],
fill="#555555")
        log_values = [math.log10(max(value, 1e-20)) for
value in values]

```

```

        high, low = max(log_values), min(log_values)
        points = []
        for point, value in enumerate(log_values):
            x = x0 + point * (x1 - x0) / max(1, len(values)
- 1)
            y = y0 + 8 + (high - value) * (y1 - y0 - 16) /
max(1e-9, high - low)
            points.extend((x, y))
        canvas.create_line(points, fill="#1479c9", width=2,
smooth=True)
        canvas.create_text(x0 + 4, y0 + 8, text=f"{10 **
high:.1e}", anchor="nw", font=("Consolas", 8))
        canvas.create_text(x0 + 4, y1 - 16, text=f"{10 **
low:.1e}", anchor="sw", font=("Consolas", 8))

    def check_hibp(self) -> None:
        if not self.password:
            self.analyze()
            digest = hashlib.sha1(self.password.encode("utf-
8")).hexdigest().upper()
            request =
urllib.request.Request(f"https://api.pwnedpasswords.com/range/{d
igest[:5]}", headers={"User-Agent": "PasswordLab"})
            try:
                with urllib.request.urlopen(request, timeout=10) as
response:
                    rows = response.read().decode("utf-
8").splitlines()
                    found = next((row for row in rows if
row.startswith(digest[5:])), None)
                    except OSError as error:
                        messagebox.showwarning("НИБП", f"Не удалось
выполнить сетевую проверку: {error}")
                        return
                    if found:
                        messagebox.showwarning("НИБП", f"Пароль найден в
утечках: {found.split(':')[1]} раз. Использовать его нельзя.")
                    else:
                        messagebox.showinfo("НИБП", "Совпадений не найдено.
Полный пароль сервису не передавался.")

```

```
if __name__ == "__main__":
    PasswordApp().mainloop()
```

Результат выполнения

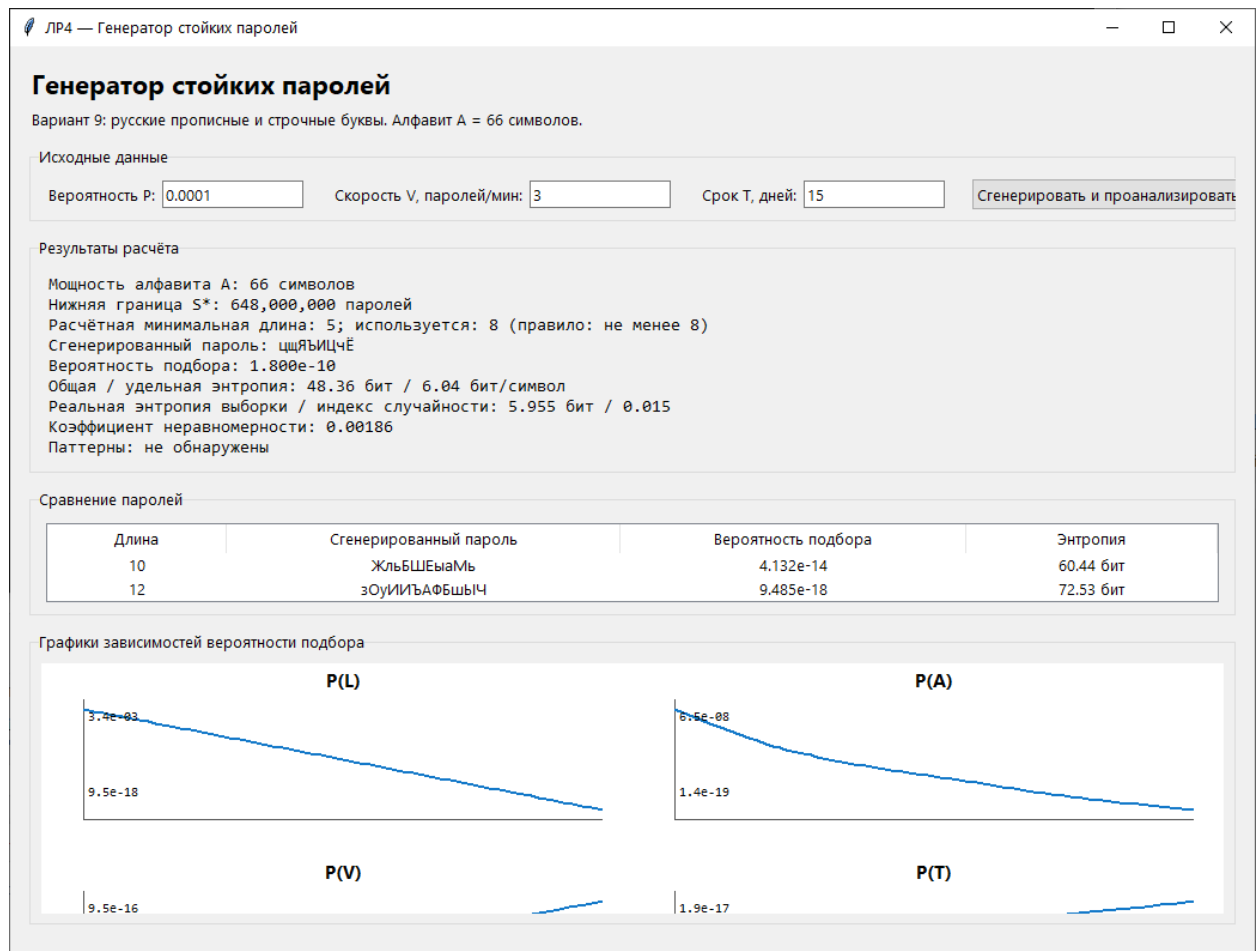


Рисунок 1 — работа генератора и результаты расчёта для варианта 9

На снимке видно нативный интерфейс программы Tkinter: она использует алфавит мощности 66, определяет $S^* = 648000000$ и формирует пароль длиной 8 символов. Общая теоретическая энтропия такого пароля равна 48,36 бит, а удельная — 6,04 бит/символ. Для каждой генерации параметры реальной выборки из 50 паролей пересчитываются, поэтому значения реальной энтропии и коэффициента неравномерности могут отличаться.

Для паролей длиной 10 и 12 символов вероятность подбора уменьшается в $66^2 = 4356$ раз при увеличении длины на два символа. Следовательно, увеличение длины существенно повышает стойкость пароля к прямому перебору.

Ответы на контрольные вопросы

1. Стойкость подсистемы идентификации и аутентификации определяется способностью противостоять получению или подбору аутентификационных данных нарушителем.
2. Пароль должен иметь длину не менее 8 символов, использовать различные группы символов и не быть словом, именем или другой предсказуемой строкой.
3. Подсистема должна ограничивать срок действия пароля, число попыток ввода (обычно до трёх) и вводить задержку после ошибочного ввода.
4. Вероятность подбора за срок действия определяется как $P = V \times T / A^L$, где V — скорость перебора, T — время атаки, A — мощность алфавита, L — длина пароля.
5. Вероятность уменьшается увеличением мощности алфавита A и длины L , а также сокращением срока действия пароля T .
6. Энтропия — мера неопределённости и случайности. Общая энтропия $E = L \times \log_2(A)$, удельная энтропия равна $\log_2(A)$; они позволяют количественно оценить пространство перебора.
7. Паттерны важны, потому что последовательности, повторы и известные слова сокращают фактическое пространство поиска и делают пароль предсказуемым.

Вывод

Разработан генератор паролей с графическим интерфейсом для варианта 9. Он генерирует пароль из русских прописных и строчных букв, выполняет расчёт стойкости, энтропийный анализ, проверку паттернов и визуализирует зависимости вероятности подбора. Использование длины 8 вместо расчётного минимума 5 обеспечивает выполнение базового требования к паролям и снижает вероятность прямого подбора до $1,800 \times 10^{-10}$.